



TECNICO EN CIBERSEGURIDAD

Participante: Luis Mario Reynoso

Técnico: Ciberseguridad

Nombre del Módulo: CRIPTOGRAFÍA Y CIFRADO DE DATOS

Facilitador: Kevin Feliz Henríquez

Centro: Cyberclass

Nombre Actividad: Laboratorio: Cifrador y Descifrador ROT13 con Python

Descripción: En esta actividad estaremos creando un programa con el lenguaje Python llamado CryptoROT Suite, que permite transformar mensajes usando el algoritmo ROT13. El programa incluye historial, contador de operaciones, limpieza de historial, exportación a archivo TXT y una explicación de por qué el mismo proceso sirve para cifrar y descifrar.

¿Qué construiremos?

Desarrollaremos una aplicación interactiva por consola para transformar mensajes utilizando el algoritmo ROT13.

- Introducir un mensaje en texto plano o texto cifrado.
- Aplicar ROT13 para transformar el mensaje.
- Usar el mismo algoritmo para cifrar y descifrar.
- Guardar cada operación en un historial temporal.
- Visualizar el historial de resultados.
- Contar la cantidad de mensajes procesados.
- Limpiar el historial cuando sea necesario.
- Exportar el historial a un archivo TXT.
- Salir del programa mediante un menú interactivo.

Objetivo del proyecto

Comprender el funcionamiento de la criptografía por sustitución y crear una aplicación interactiva en Python usando funciones, ciclos, condicionales y manipulación de caracteres ASCII.

Introducción

ROT13 es un algoritmo derivado del Cifrado César. Su nombre significa rotación de 13 posiciones. Esto quiere decir que cada letra se cambia por otra ubicada 13 espacios más adelante en el alfabeto.

La característica más importante de ROT13 es que el mismo algoritmo sirve tanto para cifrar como para descifrar. Esto ocurre porque el alfabeto tiene 26 letras, y $13 + 13 = 26$, por lo que al aplicar ROT13 dos veces se regresa al mensaje original.

- Texto plano: mensaje original que puede ser leído directamente. Ejemplo: SEGURIDAD.
- Texto cifrado: mensaje transformado por el algoritmo. Ejemplo: FRTHEVQNQ.
- Sustitución: proceso de reemplazar una letra por otra.
- Simetría: propiedad que permite usar el mismo proceso para cifrar y descifrar.

¿Qué es ROT13?

ROT13 es un algoritmo de sustitución monoalfabética. Cada letra del alfabeto se reemplaza por la letra que se encuentra 13 posiciones más adelante.

Original	ROT13
A	N
B	O
C	P
D	Q

E	R
F	S
G	T
H	U
I	V
J	W
K	X
L	Y
M	Z
N	A
O	B
P	C
Q	D
R	E
S	F
T	G
U	H
V	I
W	J
X	K
Y	L
Z	M

¿Por qué ROT13 sirve para cifrar y descifrar?

El alfabeto tiene 26 letras. ROT13 desplaza cada letra 13 posiciones. Si se aplica una segunda vez, se desplaza otras 13 posiciones, completando 26 posiciones y volviendo al punto inicial.

Proceso	Resultado
Mensaje original	HOLA
Aplicar ROT13 una vez	UBYN
Aplicar ROT13 otra vez	HOLA

Idea clave

Una persona que reciba un mensaje en ROT13 solo necesita aplicar ROT13 nuevamente para obtener el mensaje original. ROT13 no utiliza clave secreta, por eso no es seguro para proteger información real.

FASE 1: Preparación del entorno

Paso 1: Instalar Python

Python es un lenguaje de programación sencillo, poderoso y utilizado en ciberseguridad, automatización, inteligencia artificial, ciencia de datos y desarrollo web.

Descarga recomendada: <https://www.python.org>

Paso 2: Instalar Visual Studio Code

Visual Studio Code es un editor de código que permite escribir, guardar y ejecutar programas en Python.

Descarga recomendada: <https://code.visualstudio.com>

Paso 3: Instalar la extensión de Python

Dentro de Visual Studio Code, se debe abrir la sección Extensions, buscar Python e instalar la extensión publicada por Microsoft.

FASE 2: Creación del proyecto

Abrir CMD o PowerShell y ejecutar los siguientes comandos:

```
cd Desktop
mkdir rot13
cd rot13
code .
```

La estructura del proyecto será la siguiente:

```
rot13/
├── app.py
```

FASE 3: Comprendiendo el programa

Variable global: historial

```
historial = []
```

La lista historial se utiliza para guardar temporalmente los mensajes procesados por el programa. Cada vez que el usuario transforma un mensaje, se guarda el mensaje original y su resultado.

Ejemplo de un registro guardado:

```
{
  "original": "HOLA",
  "resultado": "UBYN"
}
```

Variable global: contador_operaciones

```
contador_operaciones = 0
```

Esta variable almacena la cantidad total de mensajes procesados durante la ejecución del programa. Cada vez que se transforma un mensaje, el contador aumenta en 1.

Función mostrar_menu()

```
def mostrar_menu():
    print("\n===== CRYPTOROT SUITE =====")
    print("1. Transformar mensaje con ROT13")
    print("2. Ver historial")
    print("3. Ver contador de operaciones")
    print("4. Limpiar historial")
    print("5. Exportar historial a TXT")
    print("6. Salir")
```

Esta función muestra las opciones principales del programa. El menú permite transformar mensajes, ver el historial, consultar el contador, limpiar registros, exportar datos y salir de la aplicación.

Función aplicar_rot13(texto)

```
def aplicar_rot13(texto):
```

Esta función recibe un texto y devuelve el resultado de aplicar ROT13. Puede recibir texto plano o texto cifrado, porque el mismo proceso sirve para ambas operaciones.

```
    resultado = ""
```

La variable resultado almacena el mensaje transformado.

```
    for caracter in texto:
```

El ciclo for recorre cada carácter del texto ingresado por el usuario.

```
    if caracter.isalpha():
```

Esta condición verifica si el carácter actual es una letra. Si no es una letra, se mantiene igual.

```
    if caracter.isupper():
        codigo_base = 65
    else:
        codigo_base = 97
```

Estas líneas permiten conservar el formato original del texto. Si la letra es mayúscula, se usa el código ASCII 65. Si es minúscula, se usa el código ASCII 97.

```
posicion = ord(caracter) - codigo_base
```

Aquí se convierte la letra en una posición del alfabeto. Por ejemplo, A equivale a 0, B equivale a 1 y C equivale a 2.

```
nueva_posicion = (posicion + 13) % 26
```

Esta línea aplica el desplazamiento de 13 posiciones. El módulo 26 permite volver al inicio del alfabeto cuando se supera la letra Z.

```
resultado += chr(nueva_posicion + codigo_base)
```

Finalmente, se convierte la nueva posición numérica en una letra y se agrega al resultado.

Función guardar_historial(mensaje_original, resultado)

```
def guardar_historial(mensaje_original, resultado):
    historial.append({
        "original": mensaje_original,
        "resultado": resultado
    })
```

Esta función guarda en memoria el mensaje original y el resultado obtenido después de aplicar ROT13. El método append() agrega un nuevo registro al final de la lista historial.

Función mostrar_historial()

```
def mostrar_historial():
```

Esta función muestra todos los registros guardados en el historial.

```
if len(historial) == 0:
    print("\nNo existen registros en el historial.")
```

Si no existen operaciones guardadas, el programa muestra un mensaje indicando que el historial está vacío.

```
for indice, registro in enumerate(historial, start=1):
```

El ciclo for recorre cada registro del historial. La función enumerate() permite mostrar un número para cada registro, iniciando desde 1.

Función mostrar_contador()

```
def mostrar_contador():
    print(f"\nTotal de mensajes procesados: {contador_operaciones}")
```

Esta función muestra la cantidad total de mensajes que han sido procesados durante la ejecución del programa.

Función limpiar_historial()

```
def limpiar_historial():  
    historial.clear()  
    print("\nHistorial limpiado correctamente.")
```

Esta función elimina todos los registros guardados dentro de la lista historial. El método clear() vacía la lista sin necesidad de crear otra nueva.

Función exportar_historial()

```
def exportar_historial():
```

Esta función permite guardar el historial en un archivo llamado historial_rot13.txt.

```
    archivo = open("historial_rot13.txt", "w", encoding="utf-8")
```

La función open() crea o sobrescribe el archivo de texto. El modo "w" significa escritura.

```
    archivo.write("==== HISTORIAL CRYPTOROT SUITE =====\n")
```

El método write() permite escribir información dentro del archivo TXT.

```
    archivo.close()
```

El método close() cierra el archivo después de terminar la escritura.

Función transformar_mensaje()

```
def transformar_mensaje():  
    global contador_operaciones
```

Esta función controla el proceso principal de transformación ROT13. Usa global contador_operaciones para poder modificar la variable global del contador.

```
    mensaje = input("\nDigite el mensaje: ")  
    resultado = aplicar_rot13(mensaje)
```

Primero solicita el mensaje al usuario y luego llama a la función aplicar_rot13() para transformarlo.

```
    guardar_historial(mensaje, resultado)  
    contador_operaciones += 1
```

Después guarda el resultado en el historial y aumenta el contador de operaciones en 1.

Función principal()

```
def principal():  
    while True:  
        mostrar_menu()
```

La función principal controla todo el flujo del programa. El ciclo while True mantiene el programa funcionando hasta que el usuario seleccione la opción de salir.

```
opcion = input("Seleccione una opcion: ").strip()
```

Aquí el usuario escribe la opción que desea ejecutar. El método strip() elimina espacios al principio y al final de la entrada.

```
elif opcion == "6":
    print("\nGracias por usar CryptoROT Suite.")
    return
```

Cuando el usuario selecciona la opción 6, se muestra un mensaje de despedida y return finaliza la función principal. Al terminar la función principal, el programa se cierra.

Código completo del programa

Este es el código completo de CryptoROT Suite. Incluye transformación ROT13, historial, contador de operaciones, limpieza de historial, exportación a TXT y menú interactivo.

```
# =====
# Proyecto: CryptoROT Suite
# Descripcion:
# Aplicacion de consola para transformar mensajes
# usando el algoritmo ROT13.
# =====
```

```
historial = []
contador_operaciones = 0
```

```
def mostrar_menu():
    print("\n==== CRYPTOROT SUITE ====")
    print("1. Transformar mensaje con ROT13")
    print("2. Ver historial")
    print("3. Ver contador de operaciones")
    print("4. Limpiar historial")
    print("5. Exportar historial a TXT")
    print("6. Salir")
```

```
def aplicar_rot13(texto):
    resultado = ""

    for caracter in texto:
        if caracter.isalpha():

            if caracter.isupper():
```



```

        codigo_base = 65
    else:
        codigo_base = 97

    posicion = ord(caracter) - codigo_base
    nueva_posicion = (posicion + 13) % 26
    resultado += chr(nueva_posicion + codigo_base)

else:
    resultado += caracter

return resultado

```

Código completo del programa - continuación

```

def guardar_historial(mensaje_original, resultado):
    historial.append({
        "original": mensaje_original,
        "resultado": resultado
    })

def mostrar_historial():
    if len(historial) == 0:
        print("\nNo existen registros en el historial.")
    else:
        print("\n===== HISTORIAL =====")

        for indice, registro in enumerate(historial, start=1):
            print(f"\nRegistro #{indice}")
            print(f"Original : {registro['original']}")
            print(f"Resultado: {registro['resultado']}")
            print("-----")

def mostrar_contador():
    print(f"\nTotal de mensajes procesados: {contador_operaciones}")

def limpiar_historial():
    historial.clear()
    print("\nHistorial limpiado correctamente.")

def exportar_historial():
    if len(historial) == 0:
        print("\nNo hay historial para exportar.")
    return

```

```

archivo = open("historial_rot13.txt", "w", encoding="utf-8")
archivo.write("==== HISTORIAL CRYPTOROT SUITE =====\n")

for indice, registro in enumerate(historial, start=1):
    archivo.write(f"\nRegistro #{indice}\n")
    archivo.write(f"Original : {registro['original']}\n")
    archivo.write(f"Resultado: {registro['resultado']}\n")
    archivo.write("-----\n")

```

Código completo del programa - continuación

```

archivo.close()

print("\nHistorial exportado correctamente en el archivo:")
print("historial_rot13.txt")

def transformar_mensaje():
    global contador_operaciones

    mensaje = input("\nDigite el mensaje: ")
    resultado = aplicar_rot13(mensaje)

    print("\nResultado:")
    print(resultado)

    guardar_historial(mensaje, resultado)
    contador_operaciones += 1

def principal():
    while True:
        mostrar_menu()

        opcion = input("Seleccione una opcion: ").strip()

        if opcion == "1":
            transformar_mensaje()

        elif opcion == "2":
            mostrar_historial()

        elif opcion == "3":
            mostrar_contador()

        elif opcion == "4":
            limpiar_historial()

```

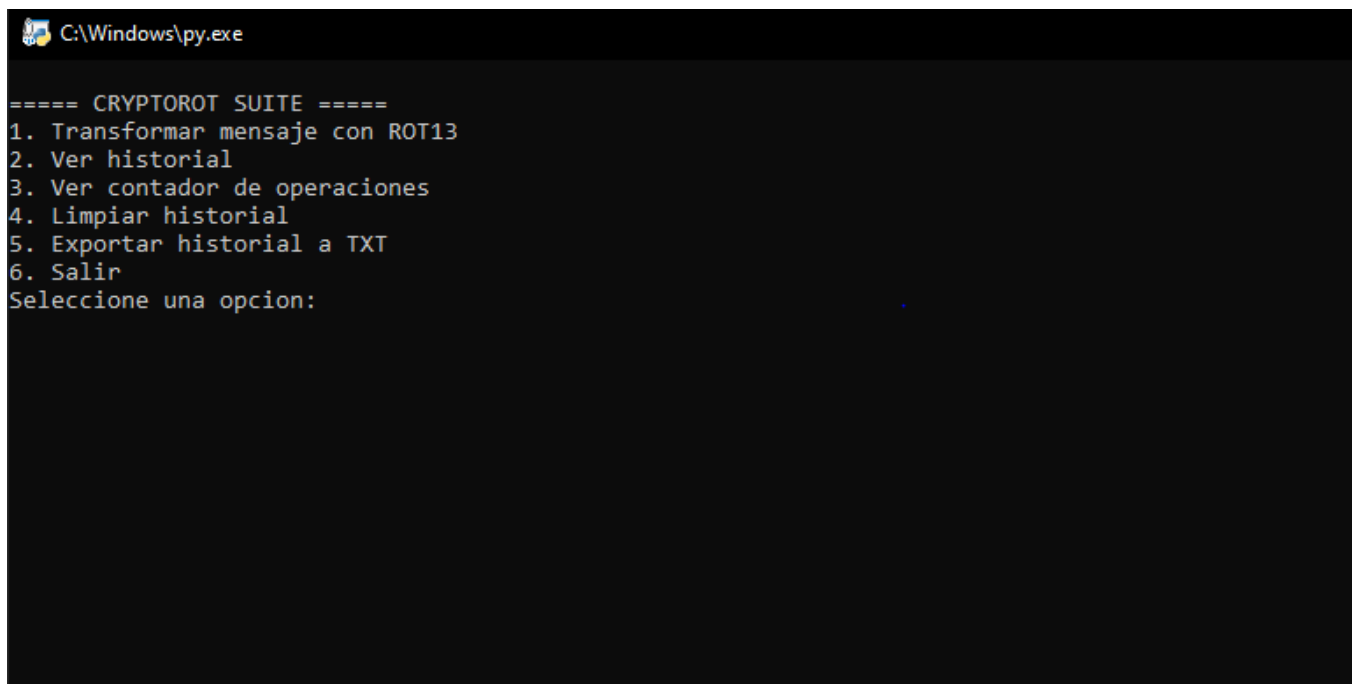
```
        elif opcion == "5":
            exportar_historial()

        elif opcion == "6":
            print("\nGracias por usar CryptoROT Suite.")

            return

        else:
            print("\nOpcion invalida. Intente nuevamente.")

principal()
```



```
C:\Windows\py.exe

===== CRYPTOROT SUITE =====
1. Transformar mensaje con ROT13
2. Ver historial
3. Ver contador de operaciones
4. Limpiar historial
5. Exportar historial a TXT
6. Salir
Seleccione una opcion: .
```

Conclusión

Este proyecto permite comprender cómo funciona un algoritmo de sustitución simple. ROT13 transforma cada letra desplazándola 13 posiciones dentro del alfabeto. Debido a que el alfabeto tiene 26 letras, al aplicar ROT13 dos veces se obtiene nuevamente el mensaje original.

Aunque ROT13 no es seguro para proteger información real, es útil para aprender conceptos básicos de criptografía, simetría, manipulación de caracteres ASCII, uso de funciones, ciclos, condicionales e interacción por consola en Python.

